

Chess AI using Monte Carlo Tree Search and Reinforcement Learning

Andrew Jones
University of Michigan
Ann Arbor, Michigan, USA
drej@umich.edu

Abstract—This paper explores the implementation on consumer hardware of a Chess AI based on Google DeepMind’s AlphaZero approach, and documents some of its challenges.

I. INTRODUCTION

Chess has long been one of the frontiers for Artificial Intelligence. In the 1990s, IBM developed a supercomputer named **Deep Blue**, which defeated the world champion Garry Kasparov. Deep Blue’s software was meticulously crafted, with an evaluation function pre-programmed using hundreds of thousands of games and positions.

For about twenty years afterward, chess engines followed a similar approach, relying on hand-crafted evaluation functions. However, in the mid-2010s, Google DeepMind introduced **AlphaZero**, a multipurpose engine capable of playing Go, Shogi, and Chess. AlphaZero used a novel neural-network-based architecture and learned entirely through self-play using reinforcement learning. It beat Stockfish, the best traditional chess engine of the time. Today, even Stockfish has transitioned away from hand-crafted evaluation functions toward reinforcement learning.

We attempt here to emulate the AlphaZero approach on consumer hardware, as outlined in their paper. [1]

II. METHOD

A. Problem Formulation

This task is framed as a reinforcement learning problem where an AI agent learns chess through self-play. The input consists of a $20 \times 8 \times 8$ tensor encoding the chess position. It includes:

- 12 planes for piece positions (6 piece types $\times$ 2 colors),
- 1 plane indicating the player’s turn,
- 4 planes for castling rights,
- 1 plane for en passant,
- 1 plane for the 50-move rule counter, and
- 1 plane for repetition detection.

The model produces two outputs:

- 1) A policy π , representing probabilities over all 1968 possible moves, and
- 2) A value v , which evaluates the current position in the range $[-1, 1]$.

B. Architecture

The implementation is organized into five layers:

1) Game Layer:

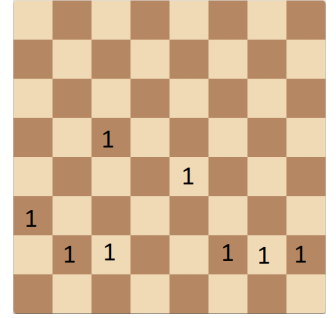
- **ChessGame**: Implements game logic and state tracking.
- **ChessReplay**: Saves and replays games in .pgn format for analysis and training.

2) Interface Layer:

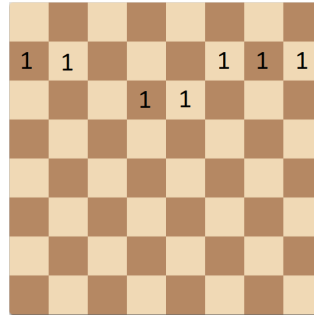
- **ChessActionSpace**: Maps chess moves to indices, ensuring compatibility with the agent’s decision-making process.
- **ChessEnvironment**: Converts board states into tensor representations and computes game-state rewards. An example can be seen in the figures below.



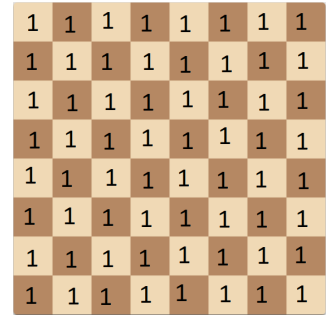
(a) Given chess position



(b) The 1s are white pawns, all other entries in this 8×8 are 0 but unshown.



(c) The 1s are black pawns.



(d) Everything being 1s indicates it’s white’s turn; otherwise, it would be all 0s.

Fig. 1: Each aspect of this chess position is given an 8×8 plane, keeping track of three-move repetition, castling rights, en passant, and piece locations.

3) Agent Layer:

- **ChessAgent**: Contains subclasses for different agent types:
 - **RandomAgent**: Makes random decisions.
 - **MCTSAgent**: Uses Monte Carlo Tree Search for move selection, guided by the policy network.
- **MCTS**: Runs simulations to balance exploration and exploitation when selecting moves.

4) *Training Layer:*

- **ExperienceReplay:** Buffers decisions, rewards, and state transitions for use in training.
 - **ChessTrainer:** Manages the training pipeline, including gameplay simulation and neural network optimization.
- 5) *Model Layer:*
- **PolicyNetwork:** A convolutional neural network processes the input tensor to output:
 - **Policy Head:** Predicts move probabilities.
 - **Value Head:** Evaluates the position's strength.

C. Training Procedure

The training process consists of the following stages:

- 1) Simulate self-play games to collect training data.
- 2) Store decisions, outcomes, and game states in the **ExperienceReplay** buffer.
- 3) Optimize the **PolicyNetwork** to predict accurate move probabilities and evaluate positions.
- 4) Train with reward functions based on game outcomes, including adjustments for intermediate results (e.g., capturing pieces).

III. RESULTS

The results of a 100 game training run of MCTSAgent vs RandomAgent can be seen in Figure 2. In this run, rewards were given each time a piece was taken proportional to the value of the piece taken, and the value head was trained on results, with -0.5 given for a draw, +3 given for a win, and -3 given for a loss. Loss remains high, and the W/L/D of 3/5/92 shows that, in short, the model has not quite learned effective chess playing. To cover a few of the challenges which have come up, which are left, and some potential directions to go next.

The biggest challenges have come in the MCTS algorithm.

Early training runs resulted in the MCTSAgent choosing repetitive moves; particularly spending most of the game moving the knight in the corner back and forth. Giving rewards for taking material (and no penalty for losing material), combined with progressive widening (a type of weighting to help ensure it doesn't overexplore the same node) reduced this greatly.

There were issues making the tree search go further than depth 1 or 2 (meaning searching across more than just one or two moves into the future). After some bugs were fixed, causing the child nodes not to consistently expand, depth 5-6+ is reached consistently in search. However, once depth was fixed, the largest problem with the algorithm was made clear: Speed.

```
Sample of states and predictions:
Game losses - Policy: 7.2002, Value: 0.0025, Overall: 7.2027

Training Progress:
Games Played: 100
Average Total Loss: 8.4526
Average Policy Loss: 7.7938
Average Value Loss: 0.6588
Win Rate: 3.0%
W/L/D: 3/5/92
Checkpoints saved: checkpoints\checkpoint_100.pt and checkpoints\checkpoint.pt

Training completed!
Final Statistics:
Games Played: 100
Final Win Rate: 3.0%
W/L/D: 3/5/92
Checkpoints saved: checkpoints\checkpoint_100.pt and checkpoints\checkpoint.pt
```

Fig. 2: Loss over training runs. The high loss and poor W/L/D ratio indicates that the model has not yet learned effective chess strategies.

Since initial implementation, the algorithm has been sped up about 25x, but still takes 16 hours to run 100 games, which is too few to see serious learning. Some of the optimizations include: Maintaining a single 20x8x8 tensor and updating it incrementally, to reduce tensor creation; reusing nodes when possible; and use of tensor operations over loops in many places.

An example of a profile of one loop of MCTS can be seen in Figure 3, to help find in which functions bottlenecks occur. Some optimizations which are still left to try include proper batching of nodes such that multiple can be evaluated at once, and fixing more of the loops to use direct tensor operations instead.

```

 profiling 670 search...
Profile Results:
 731879 function calls (727882 primitive calls) to 18,434 seconds

Ordered by: cumulative time
List reduced from 373 to 28 due to restriction: CBO:

ncalls  tottime  pcalltime  ctime  filename:lineno(function)
1      4.459      0.475      0.475  C:\Users\jacob\AppData\Local\Programs\Python\Python312\Lib\site-packages\torch\nn\modules\conv\conv1d.py:144(run_conv)
18081   0.382      0.045      0.045  C:\Users\jacob\AppData\Local\Programs\Python\Python312\Lib\site-packages\torch\nn\modules\conv\conv1d.py:144(run_conv)
18081   0.382      0.045      0.045  C:\Users\jacob\AppData\Local\Programs\Python\Python312\Lib\site-packages\torch\nn\modules\conv\conv1d.py:144(run_conv)
18081   0.382      0.045      0.045  C:\Users\jacob\AppData\Local\Programs\Python\Python312\Lib\site-packages\torch\nn\modules\conv\conv1d.py:144(run_conv)
2809   0.377      0.235      0.000  C:\Users\jacob\AppData\Local\Programs\Python\Python312\Lib\site-packages\torch\nn\modules\conv\conv1d.py:144(run_conv)
2809   0.377      0.235      0.000  C:\Users\jacob\AppData\Local\Programs\Python\Python312\Lib\site-packages\torch\nn\modules\conv\conv1d.py:144(run_conv)
18081   0.286      0.000      0.000  C:\Users\jacob\AppData\Local\Programs\Python\Python312\Lib\site-packages\torch\nn\modules\conv\conv1d.py:144(run_conv)
18081   0.286      0.000      0.000  C:\Users\jacob\AppData\Local\Programs\Python\Python312\Lib\site-packages\torch\nn\modules\conv\conv1d.py:144(run_conv)
18081   0.462      1.622      0.000  C:\Users\jacob\AppData\Local\Programs\Python\Python312\Lib\site-packages\torch\nn\modules\conv\conv1d.py:144(run_conv)
18081   0.462      1.622      0.000  C:\Users\jacob\AppData\Local\Programs\Python\Python312\Lib\site-packages\torch\nn\modules\conv\conv1d.py:144(run_conv)
727882  0.445      0.445      0.000  C:\Users\jacob\AppData\Local\Programs\Python\Python312\Lib\site-packages\torch\nn\modules\conv\conv1d.py:144(run_conv)
727882  0.445      0.445      0.000  C:\Users\jacob\AppData\Local\Programs\Python\Python312\Lib\site-packages\torch\nn\modules\conv\conv1d.py:144(run_conv)
6480   0.835      1.157      0.000  C:\Users\jacob\AppData\Local\Programs\Python\Python312\Lib\site-packages\torch\nn\modules\conv\conv1d.py:144(run_conv)
6480   0.835      1.157      0.000  C:\Users\jacob\AppData\Local\Programs\Python\Python312\Lib\site-packages\torch\nn\modules\conv\conv1d.py:144(run_conv)
6480   0.835      1.157      0.000  C:\Users\jacob\AppData\Local\Programs\Python\Python312\Lib\site-packages\torch\nn\modules\conv\conv1d.py:144(run_conv)
4048   0.722      0.722      0.000  C:\Users\jacob\AppData\Local\Programs\Python\Python312\Lib\site-packages\torch\nn\modules\conv\conv1d.py:144(run_conv)
4048   0.722      0.722      0.000  C:\Users\jacob\AppData\Local\Programs\Python\Python312\Lib\site-packages\torch\nn\modules\conv\conv1d.py:144(run_conv)
801    0.007      0.777      0.001  C:\Users\jacob\AppData\Local\Programs\Python\Python312\Lib\site-packages\torch\nn\modules\conv\conv1d.py:144(run_conv)
801    0.007      0.777      0.001  C:\Users\jacob\AppData\Local\Programs\Python\Python312\Lib\site-packages\torch\nn\modules\conv\conv1d.py:144(run_conv)
6480   0.000      0.582      0.000  C:\Users\jacob\AppData\Local\Programs\Python\Python312\Lib\site-packages\torch\nn\modules\conv\conv1d.py:144(run_conv)
6480   0.575      0.573      0.000  C:\Users\jacob\AppData\Local\Programs\Python\Python312\Lib\site-packages\torch\nn\modules\conv\conv1d.py:144(run_conv)

```

Fig. 3: Profile of the current implementation, showing bottlenecks in performance.

Another approach which may be fruitful is to reformat the training loop to genuinely be two MCTS agents playing each other, each training the `policy_network` in separate `ExperienceReplay` batches. Playing against the random opponent may bring too much chaos into the game for low-game counts.

IV. CONCLUSION

Chess is a hard game! Much progress has been made to emulate the AlphaZero approach on consumer hardware, but optimizations of the algorithm to make training on the order of tens of thousands of games feasible are necessary to make further progress.

REFERENCES

- [1] DeepMind et al. "Mastering the game of Go without human knowledge," arXiv, 1712.01815, Dec. 2017. [Online]. Available: <https://arxiv.org/abs/1712.01815>